

DSL za skriptovanje survival-horror scenarija

Autor: *Dimitrije Jovanović RN137/2023*

Definisanje domena

Motivacija za odabir domena

Survival-horror igre su poseban žanr video igara u kome fokus nije na brzini izvođenja akcija, istraživanju otvorenog sveta i unapređivanju veština, već na atmosferi, napetosti, progresiji slagalica i upravljanju inventarom i oružjem.

Ključni elementi ovih igara su ograničeni resursi, uslovljeni prolazi kroz prostorije, logika događaja (same puzzle koje moraju po nekom odredjenom redosledu da se izvrše), kao i dobro tempirani susreti sa neprijateljima (bili to zombiji ili neka druga "stvorenja").

Zbog toga se veliki deo logike ovih igara može posmatrati kao skup scenarija i pravila koja opisuju ponašanje sveta igre i interakciju igrača sa okruženjem.

Motivacija za izbor ovog DSL-a leži u tome da survival-horror igre prirodno zahtevaju formalizaciju pravila, stanja i događaja, što ih čini pogodnim kandidatom za takav programski jezik.

Pored svega navedenog, domen survival-horror igara mi je lično interesantan, što dodatno doprinosi razumevanju problema i osmišljavanju jezika koji odgovara potrebama domena.

Objašnjenje zahteva domena

Domen survival-horror scenarija zahteva mogućnost formalnog opisa sledećih elemenata:

- scena igrice (u vidu prostorija) i prelaza izmedju njih,
- interaktivnih objekata (vrata, ključevi, sefovi i ostali predmeti),
- stanja igre, kao što su health karaktera, inventar i ostale bitne promenljive,
- uslova pod kojima se odredjene akcije (ne)mogu izvršiti,

- događaja i okidača koji se aktiviraju ulaskom u prostoriju, interakcijom sa objektom ili ispunjenjem određenog uslova,
- susreta sa neprijateljima i njihovog ponašanja u okviru scenarija.

Svi gore navedeni zahtevi su zapravo skup pravila koja definišu tok igre i iskustvo i moraju biti formalno opisani.

Obrazloženje potrebe za implementacijom programskog jezika

Iako se deo navedenih zahteva može opisati pomoću config fajlova, takav pristup nije dovoljan za opis složenije logike koja uključuje uslovno grananje, sekvencijalno izvršavanje događaja i upravljanje stanjem igre.

Domenski specifičan programski jezik omogućava izražavanje ovakve logike kroz specijalizovane jezičke elemente prilagodjene domenu, uz mogućnost statičke provere ispravnosti scenarija pre izvršavanja.

Zbog toga je upotreba ovog DSL-a opravdana jer bi omogućavao:

- izražavanje logike kroz jezičke elemente prilagodjene domenu
- čitljivije i jasnije formulisanje ponašanja u odnosu na opšte jezike
- proveru ispravnosti scena pre izvršavanja (da li je soba spremna, unlockable i slično)
- razdvajanje opisa ponašanja igre od same implementacije engine-a.

Cilj ovog DSL-a ne bi bio implementacija kompletne igre, nego opisivanje pravila i scenarija (visoko apstraktovanje istih).

Problemi iz domena pogodni za automatizaciju

Nekoliko primera problema koje je najpogodnije rešavati pomoću programskog jezika u okviru izabranog domena:

1. Upravljanje prolazima i progresijom

- Automatizacija logike zaključanih vrata, ključeva i uslovljenih prolaza između soba.

2. Opis logike slagalica (puzzle-a)

- Definisanje uslova puzli, postepenog napretka rešavanja istih i završnih stanja.

3. Sistem događaja i okidača (trigger-a)

- Automatizovano reagovanje na ulazak u scenu, interakciju sa objektima ili promenu gore navedenih stanja

4. Skriptovanje susreta sa neprijateljima

- Opis pojavljivanja neprijatelja (odakle kada i kako), menjanje vrednosti resursa itd.

5. Upravljanje resursima i balansom

- Pravila za ograničenje, dodelu i potrošnju resursa kao što su hp, municija i itemi.

Ovi problemi, pored toga što pripadaju domenu survival-horror igara značajno pojednostavljuju razvoj, analizu i validaciju survival-horror scenarija.

Definisanje leksike i sintakse (gramatika DSL-a)

Leksika

```
IDENTIFIER → /[a-zA-Z_][a-zA-Z0-9_]*/ ;
STRING_LIT → /"[^"]*" / ;
NUMBER_LIT → /[0-9]+/ ;
BOOL_LIT → "true" | "false" ;
```

Napomena o blokovima

```
INDENT → <indentation>
DEDENT → <dedentation>
```

Sintaksa

```
program
  → declaration_list EOF
  ;

// LISTA DEKLARACIJA
```

```
declaration_list
  → declaration declaration_list
  | ε
  ;
```

```
declaration
  → item_decl
  | flag_decl
  | var_decl
  | room_decl
  ;
```

// DEKLARACIJE ENTITETA

```
item_decl
  → ITEM IDENTIFIER STRING_LIT
  ;
```

```
flag_decl
  → FLAG IDENTIFIER ASSIGN BOOL_LIT
  ;
```

```
var_decl
  → VAR IDENTIFIER ASSIGN expression
  ;
```

// DEFINICIJA SOBE (SCENE)

```
room_decl
  → ROOM IDENTIFIER STRING_LIT COLON room_block
  ;
```

```
room_block
  → INDENT room_stmt_list DEDENT
  ;
```

```
room_stmt_list
  → room_stmt room_stmt_list
```

```

    | ε
    ;

room_stmt
    → door_decl
    | object_decl
    | event_block
    ;

// VRATA I OBJEKTI

door_decl
    → DOOR IDENTIFIER ARROW IDENTIFIER door_opts_opt
    ;

door_opts_opt
    → door_opts
    | ε
    ;

door_opts
    → LOCKED
    | REQUIRES IDENTIFIER
    | LOCKED REQUIRES IDENTIFIER
    ;

object_decl
    → OBJECT IDENTIFIER STRING_LIT
    ;

// EVENT BLOKOVI (OKIDAČI)

event_block
    → ON trigger COLON event_body
    ;

event_body
    → INDENT action_list DEDENT

```

```

;

trigger
  → ENTER
  | INSPECT IDENTIFIER
  | USE IDENTIFIER ON IDENTIFIER
;

// AKCIJE

action_list
  → action action_list
  | ε
;

action
  → say_stmt
  | set_stmt
  | give_stmt
  | take_stmt
  | lock_stmt
  | unlock_stmt
  | if_stmt
;

// POJEDINAČNE AKCIJE

say_stmt
  → SAY STRING_LIT
;

set_stmt
  → SET IDENTIFIER COMMA expression
;

give_stmt
  → GIVE IDENTIFIER
;

```

```

take_stmt
    → TAKE IDENTIFIER
    ;

lock_stmt
    → LOCK IDENTIFIER
    ;

unlock_stmt
    → UNLOCK IDENTIFIER
    ;

// USLOVNO GRANANJE

if_stmt
    → IF condition COLON INDENT action_list DEDENT else_part_opt
    ;

else_part_opt
    → ELSE COLON INDENT action_list DEDENT
    | ε
    ;

// USLOVI I IZRAZI

condition
    → expression rel_op expression
    | HAS LPAREN IDENTIFIER RPAREN
    ;

rel_op
    → EQ | NEQ | LT | LE | GT | GE
    ;

expression
    → term expression_tail
    ;

```

```

expression_tail
  → ADD term expression_tail
  | SUB term expression_tail
  | ε
  ;

term
  → factor term_tail
  ;

term_tail
  → MUL factor term_tail
  | DIV factor term_tail
  | ε
  ;

factor
  → NUMBER_LIT
  | IDENTIFIER
  | LPAREN expression RPAREN
  ;

```

Definisanje semantike

1) Definisanje programske paradigme

Prilikom dizajna programskog jezika za skriptovanje survival-horror scenarija, odabrana je **imperativna paradigma** sa izraženim **event-driven** pristupom. Ovakav izbor paradigme direktno proizilazi iz prirode samog domena i načina na koji survival-horror igre funkcionišu.

Imperativna paradigma je pogodna jer omogućava jasno i sekvencijalno opisivanje akcija koje menjaju **stanje** igre. U okviru jezika, stanje se ogleda kroz promenljive, logičke flagove i inventar igrača, dok se promene tog stanja ostvaruju imperativnim naredbama kao što su dodela vrednosti, dodavanje ili oduzimanje predmeta, kao i zaključavanje / otključavanje prolaza, a redosled

izvršavanja naredbi unutar jednog događaja jeste karakteristika imperativnih jezika.

Pored imperativnog aspekta, jezik je zasnovan i na **event-driven modelu izvršavanja**. Program ne poseduje kontrolni tok u vidu petlji ili funkcija, već se ponašanje sistema definiše kao reakcija na unapred definisane događaje. Događaji kao što su ulazak u prostoriju, interakcija sa objektom ili korišćenje predmeta su okidači koji aktiviraju odgovarajuće sekvence tih imperativnih naredbi. Ovakav pristup prirodno odgovara domenu takvih igara, gde se logika sveta izvršava u zavisnosti od postupaka igrača.

Druge paradigme nisu izabrane jer se ne uklapaju prirodno u ciljani domen. Objektno-orijentisana paradigma bi zahtevala modelovanje hijerarhija klasa i metoda, što bi nepotrebno zakomplikovalo jezik bez značajnog dobitka u izražajnosti. Funkcionalna paradigma, sa fokusom na podatke koji ne mogu da se modifikuju i slično nije pogodna zbog potrebe za čestim promenama stanja igre. Logička paradigma, iako pogodna za opis pravila, nije intuitivna za sekvencijalno opisivanje događaja i akcija karakterističnih za gameplay skriptovanje.

Zbog navedenih razloga, odabrana **imperativna, event-driven paradigma** predstavlja najprirodniji i najefikasniji način za izražavanje logike survival-horror scenarija, jer omogućava jasno, čitljivo i domenski prilagodjeno opisivanje ponašanja sveta igre.

2) Definisanje strategije savladavanja okruženja

Prilikom definisanja semantike programskog jezika, odabrao sam **dinamičku i globalnu strategiju savladavanja okruženja**. Ova strategija podrazumeva da programski jezik u toku izvršavanja održava i menja globalno stanje koje je dostupno svim delovima programa, a koje se ažurira kao reakcija na događaje u igri.

U okviru jezika, okruženje se sastoji od globalnih promenljivih, logičkih flagova i inventara igrača. Ovi elementi predstavljaju trenutno stanje sveta igre i mogu se menjati tokom izvršavanja programa kroz imperativne naredbe definisane u event blokovima. Na primer, vrednosti kao što su health igrača, posedovanje određenog predmeta ili stanje vrata (zaključana/otključana) čuvaju se u globalnom okruženju i utiču na dalji tok scenarija.

Dinamička strategija je izabrana jer se većina odluka u survival-horror igrama donosi na osnovu trenutnog stanja igre, koje se menja tokom vremena. Provera

uslova, kao što su posedovanje predmeta ili vrednost promenljive, vrši se u trenutku izvršavanja događaja, a ne unapred. Na taj način je omogućeno prirodno modelovanje ponašanja sveta igre koje zavisi od postupaka igrača.

Globalna strategija savladavanja okruženja je pogodna jer svi događaji i scene u okviru jednog scenarija dele isto stanje. Promena stanja u jednoj prostoriji može uticati na ponašanje u drugim prostorijama, što je česta pojava u survival-horror igrama (uključivanje struje može powerovati ceo sprat na primer). Time se omogućava takvo ponašanje sveta igre bez potrebe za kompleksnim mehanizmima lokalnih okruženja.

Zbog navedenih razloga, kombinacija dinamičke i globalne strategije savladavanja okruženja predstavlja prirodan izbor za ovaj programski jezik, jer omogućava jednostavno, fleksibilno i domenski prilagođeno upravljanje stanjem igre.

3) Detaljno definisanje sistema tipova podataka

Sistem tipova podataka u ovom DSL-u je jednostavan i prilagođen potrebama survival-horror scenarija. Jezik ne koristi eksplicitno definisane tipove u sintaksi, već implicitnu tipizaciju i jasno razdvajanje domenskih pojmova.

Ugradjeni tipovi podataka obuhvataju celobrojne vrednosti koje se koriste za opis numeričkog stanja igre (npr. health, ammo, itd.), logičke vrednosti koje se koriste za flagove i uslovne provere (najčešće u puzzle-ovima), kao i string vrednosti koje se koriste za prikaz narativnog teksta (say). Pored ovih osnovnih tipova, u jeziku postoje i domenski entiteti kao što su sobe, vrata, objekti i predmeti, koji predstavljaju posebne kategorije identifikatora i koriste se u tačno određenim kontekstima.

Korisnik programskog jezika ne može definisati nove tipove podataka. Ovakav dizajn je nameran, jer je jezik fokusiran na jedan konkretan domen i pruža unapred definisane konstrukte koji su dovoljni za opisivanje svih potrebnih scenarija. Time se postiže veća čitljivost i smanjuje mogućnost grešaka prilikom pisanja programa.

Kontrola tipova podataka vrši se kombinacijom statičkih i dinamičkih provera. Statičke provere obuhvataju validaciju postojanja referenciranih entiteta, kao i proveru ispravne upotrebe identifikatora u odgovarajućim kontekstima. Dinamičke provere se vrše tokom izvršavanja programa i odnose se na proveru trenutnog stanja igre, kao što su posedovanje predmeta ili vrednosti promenljivih u datom trenutku.

Kastovanje izmedju tipova nije podržano, jer u okviru jezika ne postoji potreba za konverzijom izmedju različitih tipova podataka. Ovakav pristup dodatno pojednostavljuje semantiku jezika i doprinosi njegovoj sigurnosti (primer: nema poente **set health**, **"ten"** ili **unlock health**).

Jezik nije u potpunosti tip-siguran, ali se osnovne greške mogu otkriti unapred, dok se greške u logici igre otkrivaju tokom samog izvršavanja.

Implementacija domenskih zadataka

U ovom delu prikazujem primere korišćenja DSL programskog jezika za rešavanje tipičnih problema iz ovog domena (survival-horror) igara. Svaki primer problema pokazuje kako se on može izraziti pomoću jezika / koda:

1) Upravljanje prolazima i progresija

- **Problem:** Igrač ne može da napusti prostoriju dok ne pronadje odgovarajući ključ.

```
item key "Rusty Key"

room foyer "Foyer":
  door exit → hall locked requires key

  on enter:
    say "The exit door is locked."

  on inspect door:
    if not has(key):
      say "It won't open without a key."
```

2) Logika slagalica

- **Problem:** Slagalica se rešava tek kada je ispunjen odredjeni uslov.

```
flag power = false

room control "Control Room":
  object generator "Power Generator"
```

```
on inspect generator:
  say "The generator is silent."
if not power:
  flag power = true
  say "The lights turn on."
```

3) Sistem događaja i okidača

- **Problem:** Pri prvom ulasku u prostoriju pokreće se narativni događaj.

```
room corridor "Dark Corridor":
  on enter:
    say "The corridor feels unnaturally quiet."
```

4) Skriptovanje susreta sa neprijateljima

- **Problem:** Neprijatelj se pojavljuje kada igrač udje u sobu, a izlaz se zaključava.

```
room lab "Laboratory":
  door exit → hall locked

  on enter:
    say "You hear footsteps behind you."
    lock exit
```

5) Upravljanje resursima i balansom

- **Problem:** Health igrača se menja u zavisnosti od događaja.

```
var health = 100

room storage "Storage Room":
  on enter:
    set health, health - 20
    say "Something scratches you in the dark."
```